



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECP3843-01 SPECIAL TOPIC IN DATA ENGINEERING

SEMESTER 6

2025/2026

TITLE: TUTORIAL 4 AI TO PIPELINE

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NO
NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
HARINI A/P SANGARAN	A23CS0081

TABLE OF CONTENT

a. What is AI-assisted data engineering.....	3
b. What is Claude/Ai agent use in the tutorial?.....	4
c. Steps to do prompt and coding.....	7
d. Reflection.....	13

a. What is AI-assisted data engineering

AI-assisted data engineering involves using AI, specifically large language models (LLMs) and self-contained AI coding agents, to aid in the creation, upkeep, and monitoring of data pipelines. AI-powered pipelines enable developers to use natural language to describe the desired functionality, then get code generated, and be able to do a quick review and tweak the entire pipeline, from data ingestion to transformation to production publishing.

AI-powered data engineering is changing the way developers work, and existing data platforms are not designed for that process. In data engineering, there are cloud runtimes, distributed systems, and shared production storage where fast and safe iteration is not the focus of AI coding agents. Most of the legacy platforms use point-and-click interfaces and do not use AI agents to write the code.

AI coding assistants that read, write and debug the code locally have been a great asset to software engineers. If a file is edited incorrectly, this can be simply undone. Data pipelines work differently by mutating shared data in cloud storage, executing on distributed systems, and it could impact hundreds of downstream consumers' production data. Traditional data platforms can never beat the rapid AI-driven iteration that modern development workflows demand. AI-assisted data engineering does that by combining the intelligence of an AI coding agent with an execution environment that offers the same safety guarantees as version control offers for code.

Role across the pipeline

The table below shows what are the stages involved in the pipeline and AI contribution across those pipeline stages.

Pipeline Stage	AI Contribution
Ingestion	Schema inference, format detection, source validation, and automated S3 import.
Transformation	Data cleaning and reshaping in parallel branches of code.
Quality checks	Expectation checks and data quality reports generated by AI.

Debugging	Identifying pipeline failures and recommending solutions, in remote settings.
Publishing	Validating and automatically merging only verified results to production.

Table 1 : Role Across the Pipeline

The table shows how AI can be utilized in each phase of a data engineering pipeline. In the ingestion phase, AI can help to infer schemas, identify data formats, validate data sources, and automate the ingestion of data from cloud storage like Amazon S3. During the transformation phase, the AI automatically creates code to clean and reshape the data in separate sections of the pipeline, eliminating manual script creation. The quality checking stage takes advantage of AI to generate automated data quality reports and perform expectation checks on the dataset. AI is also helpful during the debugging phase, identifying pipeline problems and recommending specific solutions in a secure and contained space that won't impact production data. Lastly, at the publishing phase, AI guarantees that only confirmed outcomes are atomically combined into production, safeguarding information honestly all throughout the lifecycle of the information pipeline. Therefore, this shows that AI is not just a tool that can be applied to one phase of the data engineering lifecycle but is an active ingredient in the entire process.

b. What is Claude/Ai agent use in the tutorial?

The tutorial demonstrates Nexla, an enterprise-grade, AI-powered data integration platform purpose-built for modern data engineering workflows and AI agent orchestration. Nexla is not a standalone LLM, but it is a complete agentic data platform that brings together all the pieces of data, powered by conversational AI, automated pipeline generation, multi-LLM orchestration, and governed data product management into a single no-code system. The platform manages more than one trillion records monthly for many industries, such as healthcare, financial services and retails. It is also included in more than fourteen Gartner 2025 Hype Cycles for Cloud and Artificial Intelligence. It offers two key capabilities which is Nexla Orchestrated Versatile Agents (NOVA) handles both natural language instructions and automatically provisions connectors, transformation logic, and quality checks without coding. Another one is Express (Express.dev), a new conversational data engineering

platform that allows any user, with or without technical background to write a data flow in natural language and get a production-ready pipeline within minutes.

In the tutorial, Nexla's AI agent performs a variety of different and important roles throughout the data engineering lifecycle, all of them provide tasks that would require a lot of manual engineering work.

Automated Pipeline Generation: The AI agent receives a natural language description of the desired data workflow from the user. It uses that instruction to select the connectors to or from the sources and destinations, and maps the data fields semantically, as well as to define the transformation logic and create a pipeline that can be used in production.

Data Transformation and Semantic Mapping: The AI used by Nexla infers meaning of data fields from multiple sources that have different naming, and automatically translates them into a common data model of its own, namely Nexsets. Nexsets are human readable, AI-ready virtual data products that can be used immediately by downstream AI agents, analytical tools, or application APIs without manually mapping the data, with the ability to standardize the data across all sources on the platform.

Agentic RAG Pipeline Construction: In the use case of AI and large language models, the Nexla agent automates the creation of Retrieval Augmented Generation (RAG) pipelines. This involves ingesting structured and unstructured documents, preserving their semantic structure and document hierarchy by using agentic chunking, loading processed documents into a desired vector database, and applying multi-LLM evaluation across many LLMs to maximize retrieval accuracy and reduce hallucinations.

Governance and Access Control Enforcement: The AI agent is governed inside of clearly defined boundaries set by the Nexla MCP Gateway. All data products created on the platform are controlled by role-based access controls, data masking, field-level encryption and with detailed audit trails. The gateway dynamically builds only the tools and data products that are granted access to each agent or user, so that it does not jeopardize the data security or regulatory standards.

Monitoring and Quality Assurance: Nexla's AI agent will be constantly monitoring the data pipelines to detect any data quality loss, schema drift or operational failure. If there are problems, the platform generates context-specific, natural language diagnostics instead of the

raw log data, allowing the data teams to promptly and efficiently detect and fix issues, regardless of their expertise level.

Use case diagram

The following diagram shows how each different actor will be using the Nexla's AI Agent system.

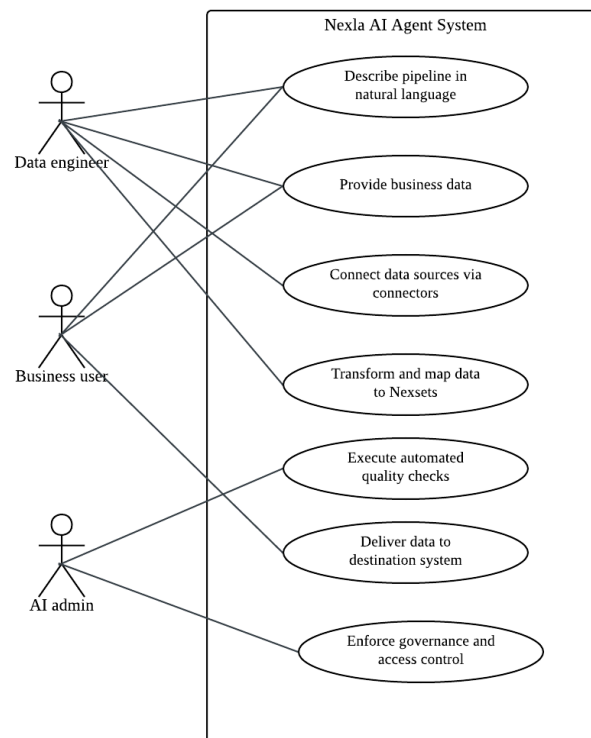
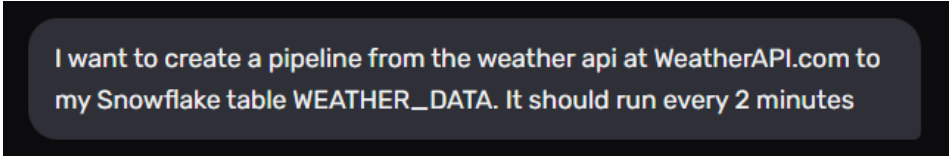


Figure 1 : Use Case Diagram

Figure 1 shows how three main actors interact with the Nexla's AI Agent System. The data engineer communicates with the system to define pipelines in natural language, to start the automated generation of pipelines, to link the data sources, and set up the data transformations. The business user is responsible for the interaction with the system via the Express conversational interface to describe workflows, to provide the business data, and consume the data products delivered by the system without having to know technical details about it. The AI admin manages the governance layer to enforce access control and set up quality control policies for the entire system.

c. Steps to do prompt and coding

Tutorial 5 (<https://www.youtube.com/watch?v=qGu7hDaJG1A> (ETL data pipeline with an AI Agent)) was followed to develop this project. The tutorial focused on building an ETL pipeline using the express.dev AI agent and the Snowflake platform as the database and data warehouse. A new database named 'tutorial' was created in Snowflake to store the extracted weather data. Following the steps demonstrated in the YouTube tutorial, WeatherAPI.com was selected as the data source to provide sample weather data for the ETL pipeline. Figure 2 shows the prompt provided to express.dev to generate the pipeline.



I want to create a pipeline from the weather api at WeatherAPI.com to my Snowflake table WEATHER_DATA. It should run every 2 minutes

Figure 2 : Prompt Given to Generate an ETL Pipeline

In response to the prompt, express.dev executed several background processes to identify the required source and destination connectors. After creating an account on WeatherAPI.com, an API key was generated to enable access to the weather data. To establish the source connection, express.dev generated a form for the user to enter the API key. Similarly, a second form was generated to collect the user's Snowflake credentials, allowing the platform to configure the destination connection. Figure 3 shows the form used to capture the WeatherAPI.com credentials, while Figure 4 shows the form used to collect the Snowflake connection details.

Express

I'll help you create a pipeline from WeatherAPI.com to your Snowflake table. Let me start by getting the connector instructions for both the source and sink.

 Loading Connector... Completed >

Express

I found that Nexla has an OpenWeather connector, but you mentioned WeatherAPI.com. Let me search for that specific connector or use the generic REST connector if needed.

 Loading Connector... Completed >

Express

Perfect! I now have the connector instructions for both REST (for WeatherAPI.com) and Snowflake. Let me gather the necessary information to set up your pipeline. I'll need to create credentials for both connectors, then create the source and sink.

Let me start by collecting information for the WeatherAPI.com credential:

Configure WeatherAPI.com Credential

Enter your WeatherAPI.com API credentials

Credential Details

* Credential Name

WeatherAPI.com Credential

* API Key

Enter your WeatherAPI.com API key 

 Cancel

 Create Credential

Express

I've created a form to collect your WeatherAPI.com credentials. Please enter your API key to proceed.



Figure 3 : Form Created by express.dev to Gather WeatherAPI.com API Key Information

Express

Great! Now let me collect the Snowflake credential information:

Preparing Form...Completed

Configure Snowflake Credential

Enter your Snowflake database credentials

Credential Details

* Credential Name

Snowflake Credential

* Host

e.g., account_name.snowflakecomputing.com

* Port

443

* Username

* Password

* Database Name

* Warehouse Name

* Schema Name

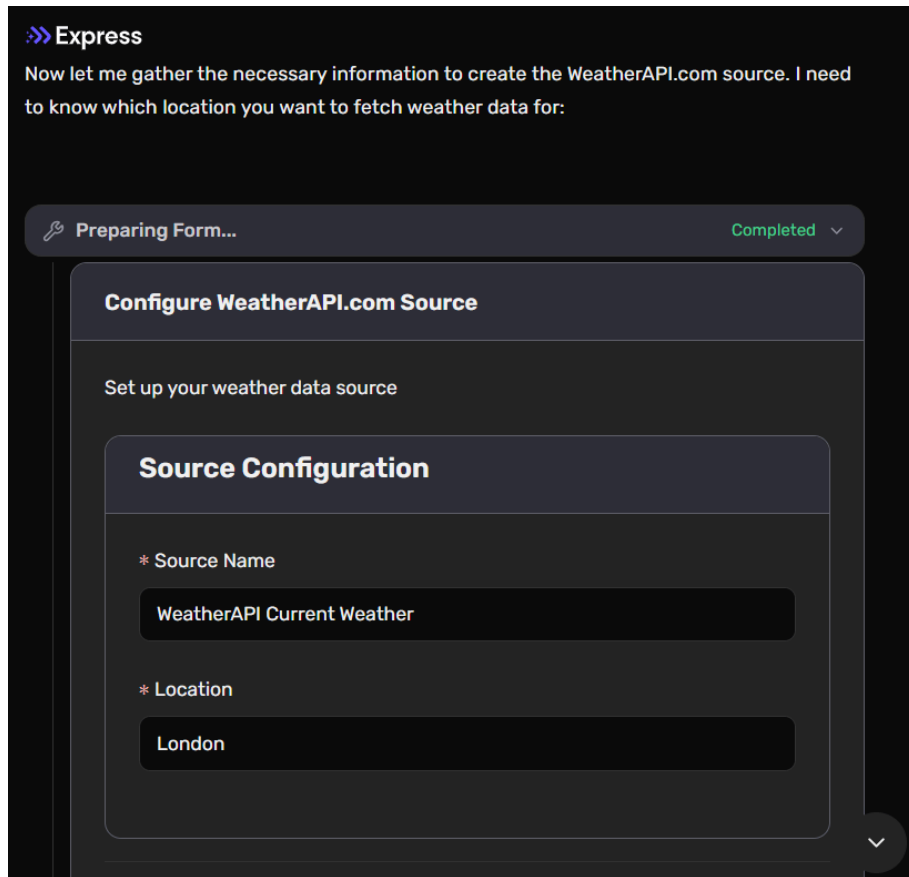
Query Tag (Optional)

e.g., nexla_integration

CancelCreate Credential

Figure 4 : Form Created by express.dev to Gather User's Snowflake Application Information

After the required credentials were provided, express.dev continued the pipeline configuration process by generating another form to collect the target weather location. Figure 5 illustrates the form used for this purpose. Based on the tutorial requirements, London was selected as the target location for weather data extraction.



The screenshot shows a web interface for configuring a weather data source. At the top, there's a header with the 'Express' logo and a message: 'Now let me gather the necessary information to create the WeatherAPI.com source. I need to know which location you want to fetch weather data for:'. Below this is a progress bar with 'Preparing Form...' and a 'Completed' status. The main section is titled 'Configure WeatherAPI.com Source' and contains the instruction 'Set up your weather data source'. Underneath is a 'Source Configuration' box with two fields: '* Source Name' with the value 'WeatherAPI Current Weather' and '* Location' with the value 'London'. A dropdown arrow is visible at the bottom right of the configuration box.

Figure 5 : Form Created by express.dev to Gather Target Weather Location

Once all necessary information had been supplied, express.dev automatically generated the ETL pipeline. During this process, a table named 'weather_data' was created within the 'tutorial' database to store the incoming weather records. Figure 6 shows the AI agent executing the necessary commands to generate the pipeline, while Figure 7 presents the SQL statement executed in Snowflake to create the weather_data table.

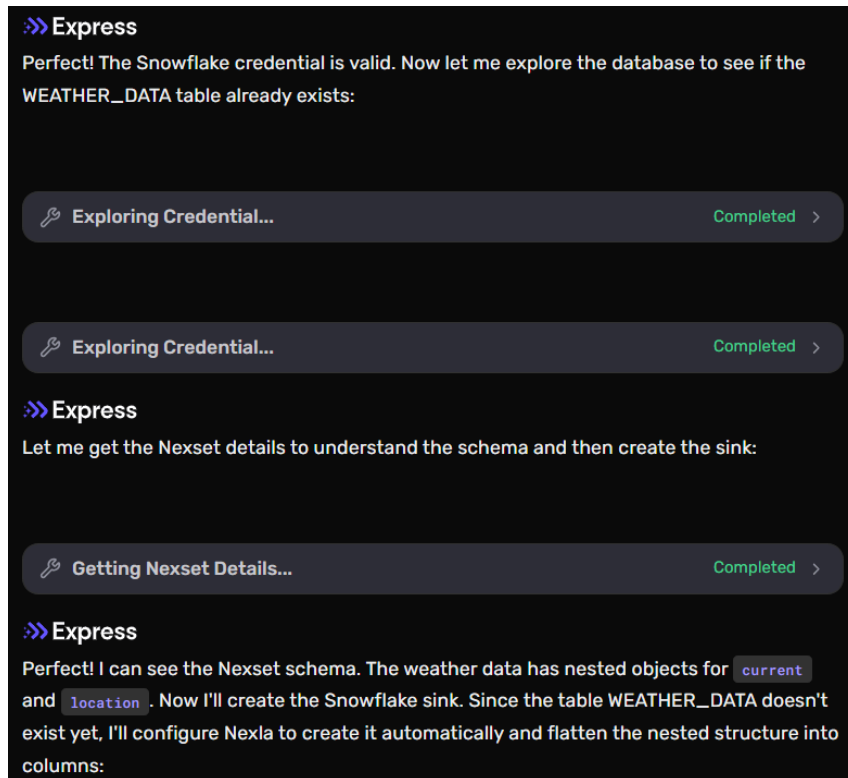


Figure 6 : *express.dev* Running Commands to Generate the ETL Pipeline

```

1  create or replace TABLE TUTORIAL.PUBLIC.WEATHER_DATA (
2      LOCATION_NAME VARCHAR(16777216),
3      LOCATION_REGION VARCHAR(16777216),
4      LOCATION_COUNTRY VARCHAR(16777216),
5      LATITUDE FLOAT,
6      LONGITUDE FLOAT,
7      TIMEZONE VARCHAR(16777216),
8      LOCAL_TIME TIMESTAMP_NTZ(9),
9      LAST_UPDATED TIMESTAMP_NTZ(9),
10     TEMP_C FLOAT,
11     TEMP_F FLOAT,
12     IS_DAY NUMBER(38,0),
13     CONDITION_TEXT VARCHAR(16777216),
14     CONDITION_ICON VARCHAR(16777216),
15     CONDITION_CODE NUMBER(38,0),
16     WIND_MPH FLOAT,
17     WIND_KPH FLOAT,
18     WIND_DEGREE NUMBER(38,0),
19     WIND_DIR VARCHAR(16777216),
20     PRESSURE_MB FLOAT,
21     PRESSURE_IN FLOAT,
22     PRECIP_MM FLOAT,
23     PRECIP_IN FLOAT,
24     HUMIDITY NUMBER(38,0),
25     CLOUD NUMBER(38,0),
26     FEELSLIKE_C FLOAT,
27     FEELSLIKE_F FLOAT,
28     VIS_KM FLOAT,
29     VIS_MILES FLOAT,
30     UV FLOAT,
31     GUST_MPH FLOAT,
32     GUST_KPH FLOAT,
33     TRANS_TEMP_F FLOAT
34 );

```

Figure 7 : SQL Statement Used to Create Table `weather_data`

Figure 8 shows the completed ETL pipeline generated by `express.dev`. The pipeline is configured to automatically collect and store weather data for London every two minutes through a scheduled Cron Job. A manual Run Now option is also available for testing and on-demand execution. Using the stored WeatherAPI credentials, the pipeline extracts the latest weather information and passes the data through a NextSet component, which manages the workflow execution. Before the data is stored, a transformation step converts temperature values into the required unit format to ensure consistency and usability. The processed data is then loaded into Snowflake, where it can be used for historical analysis, reporting, and visualization. By executing every two minutes, the pipeline provides near real-time weather updates while demonstrating an automated and scalable ETL workflow.

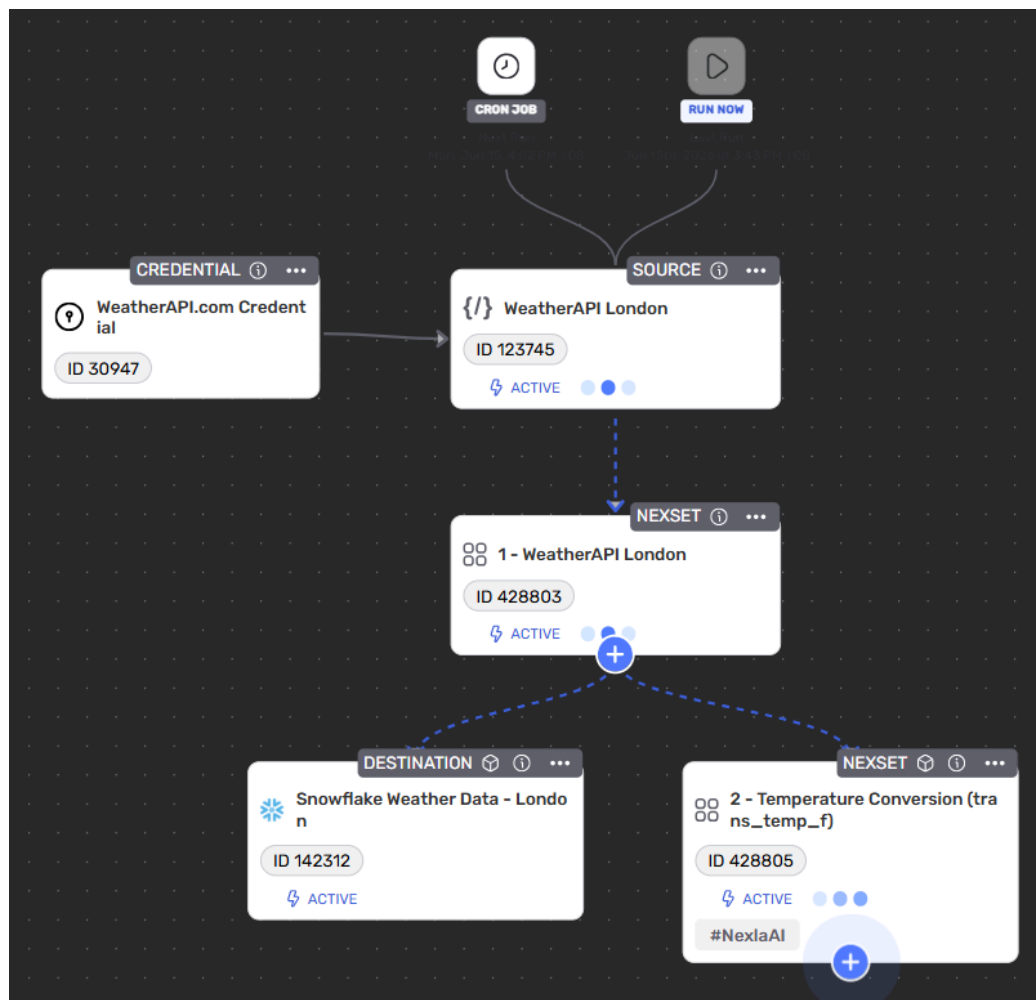


Figure 8 : ETL Pipeline Generated by `express.dev`

Finally, Figure 9 shows the records successfully extracted and loaded into the `weather_data` table through the generated ETL pipeline. The presence of these records confirms that the extraction, transformation, and loading processes were executed successfully.

TUTORIAL / PUBLIC / WEATHER_DATA

Table ACCOUNTADMIN 21 hours ago 167 10.0KB

Table Details Columns Data Preview Copy History Data Quality PREVIEW Lineage

COMPUTE_WH 100 of 167 Rows • Updated just now

	LOCATION_NAME	LOCATION_REGION	LOCATION_COUNTRY	LATITUDE	LONGITUDE	TIMEZONE	
8	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T15
9	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T15
10	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T15
11	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T15
12	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T15
13	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
14	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
15	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
16	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
17	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
18	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
19	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
20	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T16
21	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T17
22	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T17
23	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T17
24	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T17
25	London	City of London, Greater London	United Kingdom	51.5171	-0.1062	Europe/London	2026-06-14T17

Figure 9 : Data Extracted Using the ETL Pipeline Generated by express.dev

d. Reflection

- **NABIL AFLAH BOO**

Throughout the process of exploring the Nexla tutorial and completing this report, deep insight was gained into how AI-assisted workflows are fundamentally reshaping data engineering practices. The single biggest change that has been noticed is the transition from coding to “intent” which means a person can just prompt in a natural language what needs to be done and will get a production ready pipeline. In this regard, the use of AI is easy to highlight the impact it has on the development of the pipeline, including dramatic reduction in time, the democratisation of data engineering for non-technical users, and the possibility of automatic enforcement of governance without comprising agility. However, the platform also has its drawbacks, including the fact that the quality of the outputs that it produces is very reliant on the clarity and specificity of the instructions that are fed to it, and that organizations need to invest in prompt engineering skill and governance configuration to make the platform truly powerful. The system can be enhanced by creating prompt templates specific to various domains, establish human review gates to validate generated pipelines before they go to production, and add a retrieval-augmented context layer

that could incorporate historical data quality incident knowledge for the AI agent to leverage in diagnosing new pipeline issues.

- **HARINI**

This exercise provided valuable practical exposure to developing an ETL pipeline using `express.dev` and Snowflake. The use of `express.dev` simplified the pipeline development process by automatically generating the required connectors, forms, and workflow components based on user prompts. In addition, the project demonstrated how Snowflake can function as a cloud-based data warehouse for efficient data storage and management. The integration between WeatherAPI.com, `express.dev`, and Snowflake highlighted how modern data engineering tools can automate data collection, transformation, and storage with minimal manual coding. Overall, the project strengthened understanding of ETL processes, cloud data warehousing concepts, and the practical application of AI-assisted development tools in building data pipelines. The knowledge and skills gained from this exercise will be valuable for future academic projects, internships, and professional roles involving data engineering and analytics.